

Cooking Star

회원 · 게시판 기본기와 외부 API 연동을 구현한
첫 Java / Spring 팀 프로젝트 — 이후 두 프로젝트의 개선 기준이 된 출발점

Java 21

Spring Boot 3.5

Spring Security

MyBatis

PostgreSQL

JSP · Ajax

Kakao Local · Naver Search API

송태현 · Backend Developer

uiop7000@naver.com · github.com/Taehyun-0502/cooking_star

2 인 팀 · 2026.05.08 – 05.26
요리 커뮤니티 웹 서비스

목차 — 담당 범위 → 아키텍처 → 구현 → 한계와 다음

제목이나 페이지를 누르면 이동하고, 모든 장 오른쪽 아래의 '🔍 목차' 로 돌아옵니다.

01

PROJECT — 담당 범위 · 아키텍처

2 인 팀 · Git blame 6 개 도메인 100%

[03 - 04 →](#)

02

IMPLEMENT — 인증 · 권한 · 예방 설계 · 외부 API

Principal · GREATEST · ON CONFLICT · 외부 API

[05 - 07 →](#)

03

VALIDATION — 확인한 것 · 한계 · 다음 프로젝트에서 개선한 것

자동화 테스트 부족 · CSRF 비활성

[08 →](#)

시간이 3 분뿐이라면

이 세 장만 보셔도 됩니다. 제목을 누르면 이동합니다.

03

[담당 범위 한 장](#)

04

[아키텍처](#)

08

[한계 → 다음 개선](#)

회원 · 인증과 댓글 · 좋아요 · 북마크 · 팔로우, 외부 API · 관리자 기능 담당

요리 기록 게시판은 공동 구현했습니다. Naver Shopping · 장바구니 · 구매 · Gemini · 레시피 파일은 팀원 담당입니다.

서비스 전체 범위

- 레시피 조회 · 등록 · 요리 기록 (공동) · 댓글 · 좋아요 · 북마크 · 팔로우
- Kakao Local · Naver Blog (담당) · Naver Shopping · 장바구니 · 구매 · Gemini (팀원)
- 관리자 · 방문 통계

1 회원 · 인증

가입 검증 · 아이디 중복 확인 · BCrypt · Form Login · Remember-me · MEMBER/MANAGER/ADMIN 역할별 URL

2 댓글 · 좋아요

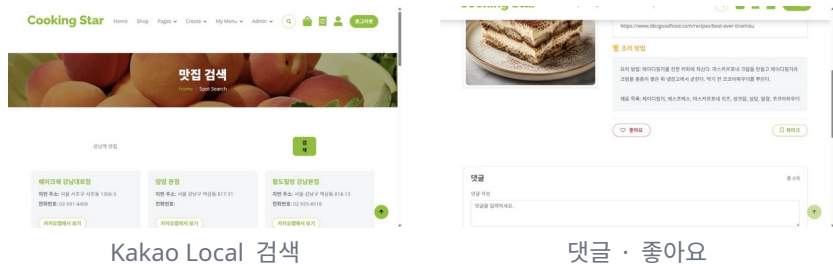
작성자 조건 삭제 · Ajax 토글 · 상태 조회 후 insert/delete · 집계 하한 (GREATEST)

3 Kakao · Naver API

장소 · 블로그 검색 · 응답 파싱 · DTO 변환 · 저장 상태 결합 (HashSet) · 설정값 주입

4 관리자 · 방문 집계

대시보드 count · 회원 / 레시피 목록 · 역할 변경 · 세션 키 + ON CONFLICT 중복 방지



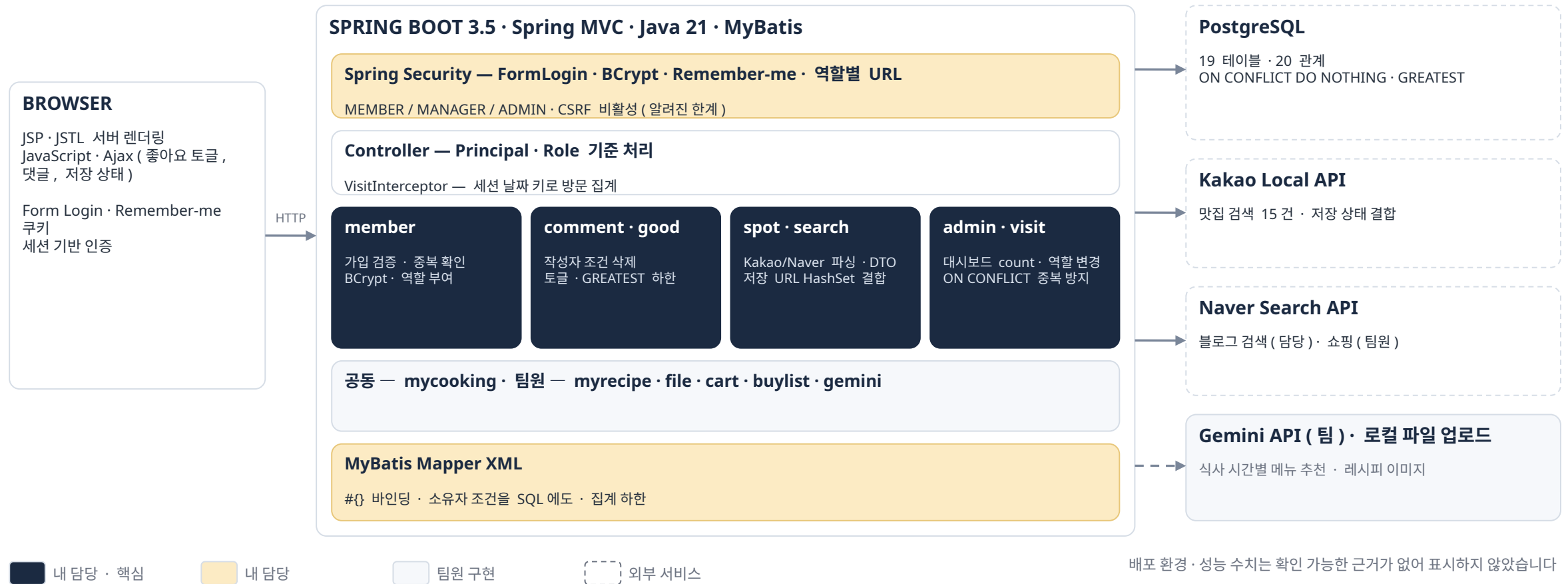
화면 : 로컬 예시 데이터로 재구동한 실제 JSP 화면

근거

Git blame — COMMENT · BOOKMARK · FOLLOW · MY_SPOT · COOK_SEARCH · COOK_VISIT 100%,
RECIPE_GOOD 91% 본인 작성

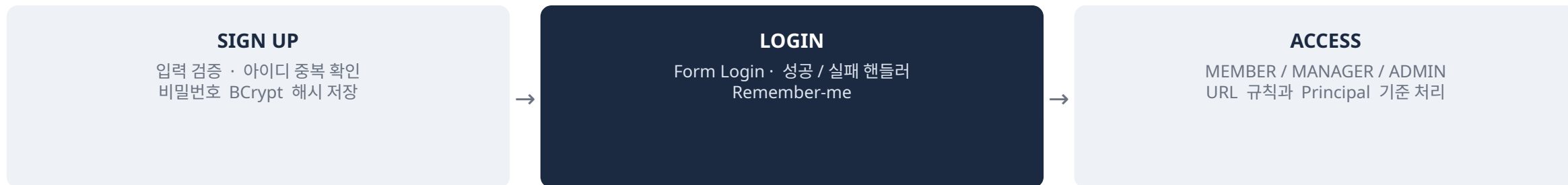
Spring MVC 계층 구조 — 인증 · 회원 · 댓글 · 외부 API 연동

요청 → Controller(Principal · Role) → Service(검증 · 외부 API · DTO) → MyBatis Mapper → PostgreSQL. 배포 · 성능 수치는 근거가 없어 표시하지 않았습니다.



클라이언트 입력보다 서버의 인증 · 권한 정보를 기준으로 판단했습니다

회원가입부터 역할별 접근 제어까지 인증 흐름을 하나의 기준으로 연결했습니다 .



```
// SecurityConfig - 역할별 URL 접근 제어 ( 요약 )
.requestMatchers("/admin/**").hasAnyRole("ADMIN", "MANAGER")
.requestMatchers("/member/join", "/member/login").permitAll()
.anyRequest().authenticated()
// Controller - 클라이언트가 보낸 username 대신 Principal 사용
String me = principal.getName();
```

SECURITY NOTE — 적용 범위

세션 기반 Form Login 으로 인증 · 인가를 구현했습니다 . CSRF 비활성화는 별도 보완 과제이며 , JWT 전환 자체가 CSRF 방어를 대신하지는 않습니다 .

다음 프로젝트로 이어진 것

Haru Health 에서는 JWT 를 컨트롤러가 수동 검증하는 한계를 겪었고 , 댕댕댕에서는 Security 필터 체인에 JWT 필터를 올려 @AuthenticationPrincipal 로 받는 구조로 발전했습니다 .

소유자 조건 · 집계 하한 · 중복 처리 — 코드에 구현한 예방 설계

GREATEST 는 집계 하한만 제한합니다 . 좋아요 행과 집계값의 일치 , 방문 날짜 · 세션의 유니크 제약과 동시 요청은 추가 검증 과제입니다 .

VISIT

APPLICATION

세션에 날짜별 방문 키 기록

DATA CONDITION

PostgreSQL ON CONFLICT DO NOTHING

EFFECT

중복 삽입 생략 (제약 확인 필요)

COMMENT

APPLICATION

댓글 번호와 작성자 조건 확인

DATA CONDITION

삭제 쿼리에도 소유자 조건 반영

EFFECT

다른 사용자의 데이터 변경 차단

LIKE

APPLICATION

토글 전 현재 상태 확인

DATA CONDITION

GREATEST 로 감소 결과 하한 설정

EFFECT

집계값이 0 아래로 내려가지 않게

요약 SQL (실제 : 각 Mapper XML)

```
-- 좋아요 취소 : 집계값 음수로 내려가지 않게      -- 댓글 삭제 : 소유자 조건을 SQL 에도
UPDATE recipe SET good_cnt = GREATEST(good_cnt - 1, 0)  DELETE FROM comment
WHERE recipe_id = #{recipeId};                          WHERE comment_id = #{id} AND writer = #{me};
-- 방문 중복 삽입 생략 : 날짜 · 세션 유니크 제약 확인 필요
INSERT INTO cook_visit (visit_date, session_key) VALUES (#{d}, #{k}) ON CONFLICT DO NOTHING;
```

외부 검색 결과의 저장 상태를 HashSet 으로 비교

검색 자체보다 요청 검증 · 응답 파싱 · 저장 상태 결합에 초점을 두었습니다. 측정 수치 없이 자료구조와 시간복잡도 차이만 코드 근거로 설명합니다.

BEFORE · LIST 순차 탐색

- 검색 결과마다 저장 목록을 처음부터 다시 순차 탐색
- 결과 N 건 $\times$ 저장 M 건 = $O(N \times M)$

AFTER · HASHSET contains

- 저장 URL Set 구성 후 contains 로 상태 판단
- 평균 시간복잡도 $O(M) + O(N) = O(N + M)$

```
// SpotService - 저장 여부 결합 (commit 748514f)
Set<String> savedUrlSet = new HashSet<>();
for (Spot saved : mySpots) savedUrlSet.add(saved.getPlaceUrl());
for (Spot spot : results)
    spot.setSaved(savedUrlSet.contains(spot.getPlaceUrl()));
```

Naver Blog Search · Kakao Local

- 검색어를 API 요청으로 변환 → 응답 JSON 파싱 → DTO → 화면 데이터
- API 키 · URL 은 코드 상수가 아니라 프로퍼티 주입 · Kakao 결과는 15 건 단위 노출

한계

외부 API 타임아웃 · 재시도 정책 및 응답시간 실측 없음. 후속 프로젝트의 외부 I/O·DB 경계 분리는 커넥션 점유 개선이며, 타임아웃 정책은 별도 과제

첫 프로젝트에서 확인한 기본기와, 다음 두 프로젝트로 가져간 개선 과제

코드에 구현한 기본기와 자동 테스트 범위, 후속 프로젝트에서 다룬 내용을 구분했습니다.

● 코드로 확인한 것

- 회원 입력 · 아이디 중복 검증
- BCrypt 비밀번호 저장
- 역할별 URL 접근 제어
- 댓글 작성자 조건 삭제
- 좋아요 집계 음수 방어
- 방문 집계 중복 방지 SQL
- Kakao 저장 상태 Set 비교

● 현재 한계

- 자동화 테스트 — contextLoads 1 개
- 외부 API 타임아웃 · 재시도 정책 없음
- 오류 유형별 사용자 메시지 없음
- 좋아요 정합성 · 방문 중복의 동시 요청 미검증
- 세션 인증의 CSRF 방어 비활성
- 배포 · 성능 근거 미확인

● 다음 프로젝트에서 개선한 것

- Haru Health — 결제 · 정산 트랜잭션 경계, 알림 예외 전파 제한
- 땡땡땡 — JWT 필터 · 토큰 회전, ErrorCode · 예외 핸들러, 외부 I/O · DB 경계 분리, 수동 동시성 확인, EC2 배포
- 별도 과제 — CSRF 방어와 외부 API 타임아웃 정책

● TAKEAWAY

권한과 입력은 서버가 판단한다 — 첫 프로젝트에서 세운 기준을 다음 프로젝트의 설계 조건으로

1

서버 기준의 권한 판단

Principal · Role · SQL 소유자 조건

2

데이터 무결성 우선

ON CONFLICT · GREATEST · 작성자 조건 삭제

3

부족한 것을 기록하고 다음에 고친다

테스트 → 트랜잭션 → 동시성 → 배포로 이어진 개선

송태현 · Backend Developer

uiop7000@naver.com · github.com/Taehyun-0502/cooking_star